P.Vishwa Nandan(192473015)

Q3 – Resource Safety, Debugging and Robust Code

1. Aim

To develop a Java-based sales report generator that reads sales data from multiple store files and calculates the total sales. To understand resource leaks and compare manual resource closing using finally with **try-with-resources**.

2. Problem Description

A retail chain generates a daily sales report by reading data from multiple store files.

Sometimes, the program may fail while reading a file. If the opened files are not closed properly, the system may produce a **Too many open files** error.

To solve this problem, two resource-management approaches are compared:

1. Manual resource closing using try-catch-finally.
2. Automatic resource closing using try-with-resources.

3. Concepts Used

Resource Management: Managing resources such as files, database connections, and network connections.

Resource Leak: A situation where a program fails to release a resource after using it.

Finally Block: A block that normally executes whether an exception occurs or not. It is commonly used to close resources manually.

Try-with-Resources: A Java feature that automatically closes resources after the try block finishes, even when an exception occurs.

5. Explanation of the Program

A. Manual Closing Using Finally

In this approach, the program opens a file and reads its sales records.

The finally block closes the file reader whether the reading process succeeds or an exception occurs.

Therefore, the resources are properly released.

B. Try-with-Resources

In this approach, the resource is declared inside the try statement.

Java automatically closes the reader after the try block finishes, including when an exception occurs.

It reduces the amount of code required for resource management.

C. Intentionally Incorrect Manual Closing

In this version, reader.close() is placed after the reading loop instead of inside a finally block.

When an exception occurs during reading, the program jumps directly to the catch block.

As a result, the close() statement is skipped and the file remains open. This demonstrates a resource leak.

The TrackedReader class counts the number of readers that remain open so that the difference can be observed.

6. Expected Output

MANUAL TRY-CATCH-FINALLY VERSION

Failed reading store2.txt: Invalid sales record: CORRUPT

Daily total: 500

Open readers: 0

TRY-WITH-RESOURCES VERSION

Failed reading store2.txt: Invalid sales record: CORRUPT

Daily total: 500

Open readers: 0

INTENTIONALLY LEAKY MANUAL VERSION

Failed reading store2.txt: Invalid sales record: CORRUPT

Daily total: 500

Open readers: 1

Output Explanation

The program reads three store files.

- Store 1 contains sales of 100 and 200.
- Store 2 contains 50, an invalid record, and 70.
- Store 3 contains 80 and 120.

When the program encounters the invalid record, it skips the remaining processing of Store 2. Its partial subtotal is not added to the final report.

Therefore:

Daily Sales = 100 + 200 + 80 + 120 = **500**

Both the correct manual-finally version and try-with-resources produce the same report and leave zero readers open.

The intentionally incorrect manual version leaves one reader open because the exception prevents its close() statement from executing.

7. Debugging and Experimental Analysis

Breakpoints can be placed at the following locations in the Java program:

1. At new TrackedReader(file) to observe when a file is opened.
2. At throw new IOException(...) to identify where the reading process fails.
3. At reader.close() to check whether the closing statement executes.
4. At the finally block to observe resource cleanup.

Debugging Observation

In the incorrect manual version, when the program encounters CORRUPT, execution moves to the catch block without executing reader.close().

In the correct manual-finally version, the finally block executes and closes the reader.

In try-with-resources, Java automatically closes the reader before the exception is handled by the catch block.

Resource Comparison

Approach	Report Total	Open Readers After Execution
Manual closing using finally	500	0
Try-with-resources	500	0
Incorrect manual closing	500	1

8. Student A – Debugging and Experimental Analysis

1. How was the resource leak identified?

The program was executed with a file containing an invalid sales record.

Breakpoints were used to observe the execution flow. The incorrect manual version skipped the close() statement when an exception occurred.

The open-reader counter confirmed that one resource remained open.

2. Which file caused the failure?

The file store2.txt caused the failure because it contained the invalid record CORRUPT.

The program displayed an error message identifying the file.

3. How was logging implemented?

The program uses the following statement:

```
System.out.println("Failed reading " +
    file.getFileName() + ": " + e.getMessage());
```

It identifies the file that caused the error and displays the reason for failure.

4. What was the experimental result?

The correct manual-finally and try-with-resources versions closed all readers.

The incorrect manual version left one reader open after the exception.

9. Student B – Design and Correctness Analysis

1. Identify the resources opened and closed.

The program opens a BufferedReader for each store file.

Each reader must be closed after its sales records have been processed.

2. Why can manual closing cause resource leaks?

When close() is placed only after successful file processing, an exception can interrupt execution before the closing statement.

The resource then remains open.

A correctly implemented finally block avoids this particular problem by closing the reader even when an exception occurs.

3. How does try-with-resources guarantee closing?

Try-with-resources automatically calls the resource's close() method when the try block finishes.

It performs cleanup even when an exception occurs.

When multiple resources are declared in the same try-with-resources statement, they are closed in the reverse order in which they were declared.

4. What is a suppressed exception?

A suppressed exception occurs when an exception is thrown while closing a resource and another exception has already occurred in the try block.

The original exception remains the primary exception, while the closing exception is attached to it as a suppressed exception.

Suppressed exceptions can be accessed using:

```
for (Throwable e : exception.getSuppressed()) {
    System.out.println(e.getMessage());
}
```

This helps developers understand both the main failure and any failure that happened during resource closing.

10. Joint Discussion

When is try-with-resources not sufficient by itself?

Try-with-resources handles resource closing, but it does not solve every problem.

Additional handling is required when:

- Input files contain invalid data.
- A file is missing or unavailable.
- The system runs out of memory or storage.
- The report must be generated completely without partial or incorrect results.
- Multiple users access the same files simultaneously.

Therefore, resource management must be combined with input validation, exception handling, logging, and proper report-generation logic.

11. Alternative Approach and Complexity Analysis

Alternative: Manual Finally vs Try-with-Resources

Manual finally: The programmer must write the closing logic. If every resource is correctly handled in finally, it can also prevent resource leaks.

Try-with-resources: Java automatically closes resources. It provides cleaner code and handles closing exceptions more safely.

Time Complexity

Let n represent the total number of sales records across all store files.

Both approaches read each record once.

Therefore, the time complexity is:

O(n)

Space Complexity and File Handles

The program processes one store file at a time.

Approach	Maximum Open File Handles
Manual finally	1 at a time
Try-with-resources	1 at a time
Incorrect manual closing	Can increase up to f if all f files fail after opening
No closing at all	Can increase up to f

Here, f represents the number of store files.

Try-with-resources and correctly implemented finally use constant additional resource space, **O(1)**, because each file is closed before the next one is processed.

An incorrect implementation can leave multiple files open, resulting in **O(f)** open file handles in the worst case.

12. Test Cases

Test Case	Input Condition	Expected Result
1	All files contain valid sales records	Report generated successfully
2	One file contains an invalid record	Error logged and invalid file skipped
3	A file is missing	File-opening error displayed
4	Exception occurs during manual reading	Finally block closes the reader
5	Exception occurs during try-with-resources	Reader closes automatically

For Test Case 1, replacing CORRUPT in store2.txt with 60 produces a total of **680** in both correct implementations.

13. Conclusion

The experiment demonstrates the importance of proper resource management in Java. Manual closing can cause resource leaks if cleanup is not performed correctly. The finally block provides a way to close resources manually, while try-with-resources closes them automatically.

Therefore, try-with-resources is generally preferred because it makes Java programs cleaner, safer, and easier to maintain.